UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Assignment 1: Sudoku solver

Release Date: 27-07-2026 at 06:00

Due Date: 21-08-2026 at 23:59

Late Deadline: 22-08-2026 at 00:59

Total Marks: 419

Read the entire specification before starting with the practical.

Contents

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- **This assignment should be completed individually.**
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Background

Sudoku is a logic puzzle game where traditionally you have to place the numbers 1 to 9 into a 9x9 grid such that no numbers repeat in any row, column or 3x3 block. This idea can be expanded to a $r \times c$ version by allowing non square blocks. An example is shown below.

1	9		2			6		12			4
									10	11	
					11		8				
2	1	8	5	3	4	10					9
	10			9		5	11	2		8	
		9	3						4		
		7						6	3		
	5		6	2	12		1			4	
12					6	4	3	5	2	1	8
				11		2					7
	7	4									2
5			11		7			1		9	12

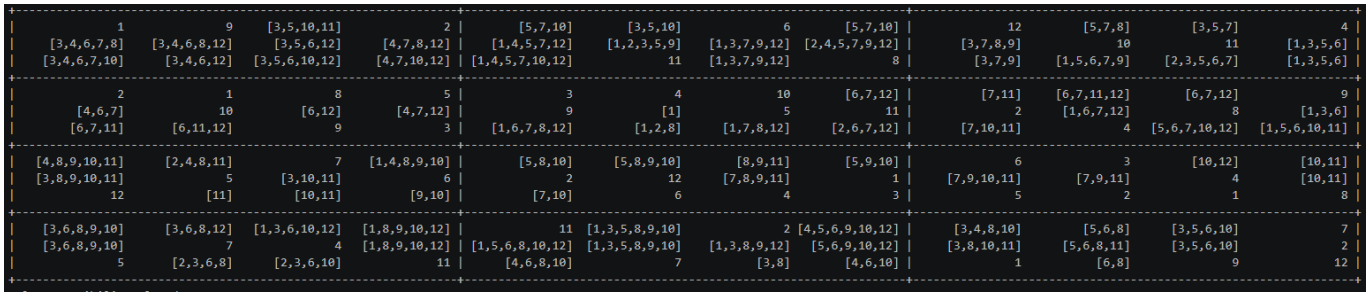
Figure 1: 3x4 Sudoku example

In this version each small block has 3 rows and 4 columns. The bigger grid is then made up of 4 rows and 3 columns of smaller blocks. The bigger grid is thus made up of 12 cells in each row and column. The value $r \times c$ will be refered to as n. Thus the goal of $r \times c$ Sudoku is to place the digits 1 to n, once each in every row, column and $r \times c$ block.

There are various Sudoku techniques that can be used to solve Sudoku grids. Some of these can be quite complex and require years of practise to easily spot. One solution is to use recursion. At every point in the puzzle, guess a random digit in the grid. If you get to a point where there are no guesses left then you can backtrack to a previous state and try a different guess. Using this method we can solve any Sudoku grid, even grids with no given digits. This is sadly a very slow approach due to the huge number of possibilities. For this assignment you will implement a $r \times c$ Sudoku solver with two manual techniques to try and cut down on the huge number of possibilities, but if these fail we will use recursion to guess a value.

3 Example

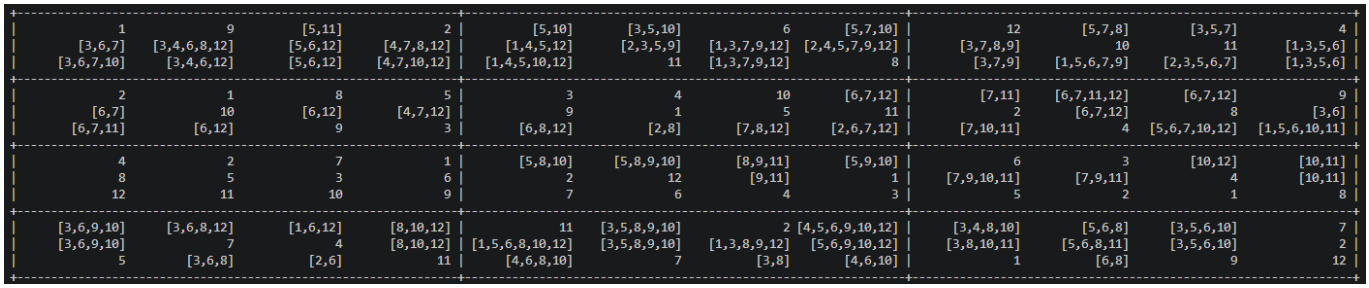
This section solves the example given in Figure 1. This example is also given in the student files and the student files shows each step, whereas this section will only explain the logic. The first step is to read in the Sudoku. This will be done using the constructor. The state of the grid is then as follows:



1	9	[3,5,10,11]	2	[5,7,10]	[3,5,10]	6	[5,7,10]	12	[5,7,8]	[3,5,7]	4
[3,4,6,7,8]	[3,4,6,8,12]	[3,5,6,12]	[4,7,8,12]	[1,4,5,7,12]	[1,2,3,5,9]	[1,3,7,9,12]	[2,4,5,7,9,12]	[3,7,8,9]	10	11	[1,3,5,6]
[3,4,6,7,10]	[3,4,6,12]	[3,5,6,10,12]	[4,7,10,12]	[1,4,5,7,10,12]	11	[1,3,7,9,12]	8	[3,7,9]	[1,5,6,7,9]	[2,3,5,6,7]	[1,3,5,6]
2	1	8	5	3	4	10	[6,7,12]	[7,11]	[6,7,11,12]	[6,7,12]	9
[4,6,7]	10	[6,12]	[4,7,12]	9	[1]	5	11	2	[1,6,7,12]	8	[1,3,6]
[6,7,11]	[6,11,12]	9	3	[1,6,7,8,12]	[1,2,8]	[1,7,8,12]	[2,6,7,12]	[7,10,11]	4	[5,6,7,10,12]	[1,5,6,10,11]
[4,8,9,10,11]	[2,4,8,11]	7	[1,4,8,9,10]	[5,8,10]	[5,8,9,10]	[8,9,11]	[5,9,10]	6	3	[10,12]	[10,11]
[3,8,9,10,11]	5	[3,10,11]	6	2	12	[7,8,9,11]	1	[7,9,10,11]	[7,9,11]	4	[10,11]
12	[11]	[10,11]	[9,10]	[7,10]	6	4	3	5	2	1	8
[3,6,8,9,10]	[3,6,8,12]	[1,3,6,10,12]	[1,8,9,10,12]	11	[1,3,5,8,9,10]	2	[4,5,6,9,10,12]	[3,4,8,10]	[5,6,8]	[3,5,6,10]	7
[3,6,8,9,10]	7	4	[1,8,9,10,12]	[1,5,6,8,10,12]	[1,3,5,8,9,10]	[1,3,8,9,12]	[5,6,9,10,12]	[3,8,10,11]	[5,6,8,11]	[3,5,6,10]	2
5	[2,3,6,8]	[2,3,6,10]	11	[4,6,8,10]	7	[3,8]	[4,6,10]	1	[6,8]	9	12

The values in square brackets are the cells that are not filled in. These arrays show all possible values for that cell. This state can be reached by starting with an empty grid, where each cell can be any value denoted by an array with values from 1 to n. Then each filled in digit is added one by one. Every time you place a digit, remove that digit as an possibility from all other cells in its row, column and block.

To start the solving process we will continuously look for cells with only one possibility and cells that are the only place a digit can go. The grid is numbered starting from the top left with a position of 0,0. The first digit our solver will place is at row 4 column 5. This is because it only has one possible value being 1. Thus we place 1. When placing this value we remove 1 as an option from all the cells in row 4, all the cells in column 5, and also all the cells in block 5. We continue filling in cells with only one option until we reach the following grid state:



1	9	[5,11]	2	[5,10]	[3,5,10]	6	[5,7,10]	12	[5,7,8]	[3,5,7]	4
[3,6,7]	[3,4,6,8,12]	[5,6,12]	[4,7,8,12]	[1,4,5,12]	[2,3,5,9]	[1,3,7,9,12]	[2,4,5,7,9,12]	[3,7,8,9]	10	11	[1,3,5,6]
[3,6,7,10]	[3,4,6,12]	[5,6,12]	[4,7,10,12]	[1,4,5,10,12]	11	[1,3,7,9,12]	8	[3,7,9]	[1,5,6,7,9]	[2,3,5,6,7]	[1,3,5,6]
2	1	8	5	3	4	10	[6,7,12]	[7,11]	[6,7,11,12]	[6,7,12]	9
[6,7]	10	[6,12]	[4,7,12]	9	1	5	11	2	[6,7,12]	8	[3,6]
[6,7,11]	[6,12]	9	3	[6,8,12]	[2,8]	[7,8,12]	[2,6,7,12]	[7,10,11]	4	[5,6,7,10,12]	[1,5,6,10,11]
4	2	7	1	[5,8,10]	[5,8,9,10]	[8,9,11]	[5,9,10]	6	3	[10,12]	[10,11]
8	5	3	6	2	12	[9,11]	1	[7,9,10,11]	[7,9,11]	4	[10,11]
12	11	10	9	7	6	4	3	5	2	1	8
[3,6,9,10]	[3,6,8,12]	[1,6,12]	[8,10,12]	11	[3,5,8,9,10]	2	[4,5,6,9,10,12]	[3,4,8,10]	[5,6,8]	[3,5,6,10]	7
[3,6,9,10]	7	4	[8,10,12]	[1,5,6,8,10,12]	[3,5,8,9,10]	[1,3,8,9,12]	[5,6,9,10,12]	[3,8,10,11]	[5,6,8,11]	[3,5,6,10]	2
5	[3,6,8]	[2,6]	11	[4,6,8,10]	7	[3,8]	[4,6,10]	1	[6,8]	9	12

At this point all cells have at least two options, thus we need another method. The second method we implement is placing digits where there is only one place for them in a row, column or block. An example of this can be seen in Row 0. In this row there is currently no 8 filled in. If you look through all the possibilities you will also see that there is only one cell in that row that has 8 as a possible value. Thus we place 8 at Row 0 Column 9.

These two basic techniques are enough to solve this Sudoku. The full steps can be seen in the provided output of the student files. For more difficult examples recursion will be used to guess values.

4 Your Task:

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
1 Cell	25
2 Sudoku utilities	65
3 Sudoku techniques	126
4 printAndSolve	170
5 Testing	33
Total	419

You are required to implement all of the functions laid out in this section. An overview of the assignment is provided by the class diagram in Figure 2.

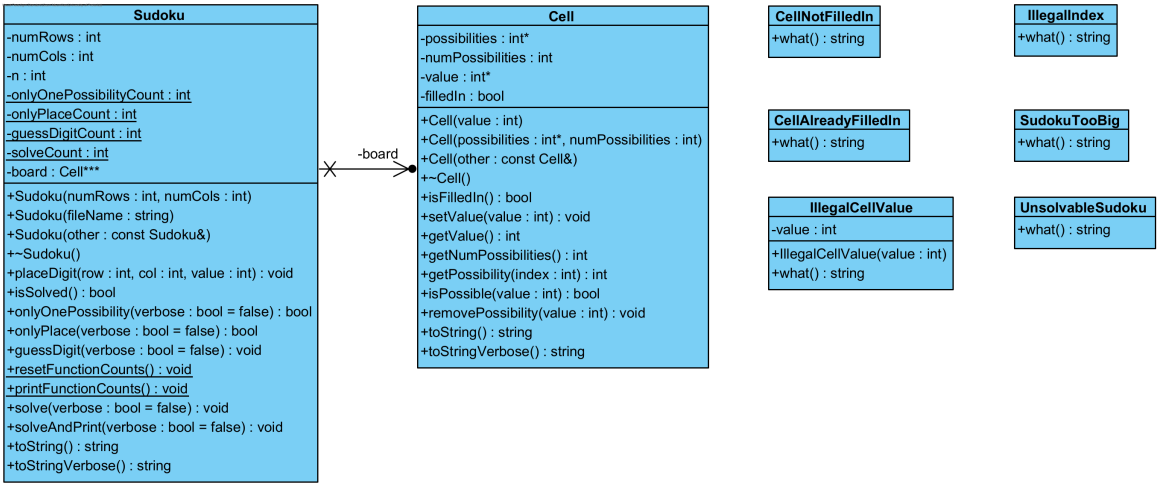


Figure 2: Class diagrams

4.1 Exceptions

The files `Exceptions.h` and `Exceptions.cpp` are given to you and will be overwritten on Fitchfork. Don't make any changes to them. There are 6 exceptions classes that will be used by the rest of the assignment. The exceptions classes are shown in Figure 3.

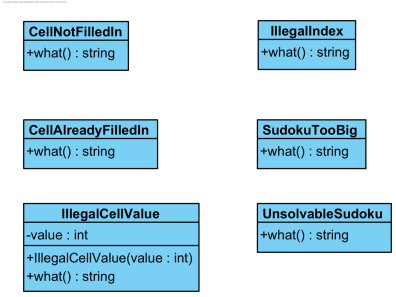


Figure 3: Exceptions UML

4.2 Cell

This class will be used to represent each cell in a sudoku grid. It will be used for both filled in and not filled in cells, and will keep track of its state using a boolean variable. The relevant functions will throw exceptions when errors occur to prevent the program from working with invalid values.

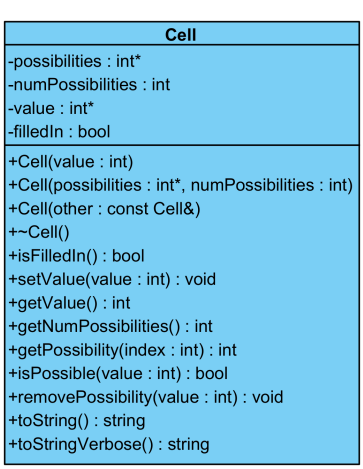


Figure 4: Cell UML

4.2.1 Members

- -possibilities: int*
  - This is an array containing all the possible values for this cell.
  - If the cell is filled in then this will be NULL.
- -numPossibilities: int
  - This is the size of the possibilities array.
  - If the cell is filled in then this will be -1.
- -value: int*
  - This is the filled in value of the cell.
  - If the cell is not filled in then this will be NULL.
- -filledIn: bool
  - This shows whether the cell is filled in.

4.2.2 Functions

- +Cell(value: int)
  - This is a constructor for the Cell class, that is used when creating a cell that is filled in.

- +Cell(possibilities: int*, numPossibilities: int)
  - This is a constructor for the Cell class, that is used when creating a cell that is not filled in.
- +Cell(other: const Cell&)
  - This is a copy constructor for the Cell class. **Remember to make deep copies.**
- +~ Cell()
  - This is the destructor for the Cell class. Remember that either possibilities **or** value has to be NULL at all times. It is not possible for both to be NULL.
- +isFilledIn(): bool
  - Accessor that returns the corresponding member variable.
- +setValue(value: int): void
  - If the cell is already filled in then throw a CellAlreadyFilledIn object.
  - If the passed-in parameter is not a possible option for this cell, then throw a IllegalCellValue object initialised with the parameter.
  - Otherwise change the member variables to indicate that the cell is now filled in with the passed-in parameter. **Remember to update all member variables.**
- +getValue(): int
  - Accessor that returns the corresponding member variable.
  - If the cell is not filled in then throw a CellNotFilledIn object.
- +getNumPossibilities(): int
  - Accessor that returns the corresponding member variable.
- +getPossibility(index: int): int
  - If the passed-in parameter is an invalid index in the array then throw an IllegalIndex object.
  - Returns the value at the passed-in index from the possibilities array.
- +isPossible(value: int): bool
  - Returns true if passed-in parameter is in the possibilities array. Returns false otherwise.
- +removePossibility(value: int): void
  - Removes the passed-in parameter from possibilities array.
  - The array should be resized such that there are no gaps in the array.

- +toString(): string
  - This function is given to you in the header file. Do not change it.
  - This gives a string representing the cell. It will be the value if it is filled in, and will be - when it is not filled in.
- +toStringVerbose(): string
  - This function is given to you in the header file. Do not change it.
  - This will give a string representing the cell. It will be the value if it is filled in, and will show the possibilities array if it is not filled in.

4.3 Sudoku

This class will be used to store and solve sudoku grids. It will use the Cell class to store the grid and will also throw exceptions when errors occur to prevent the program from working with invalid values.

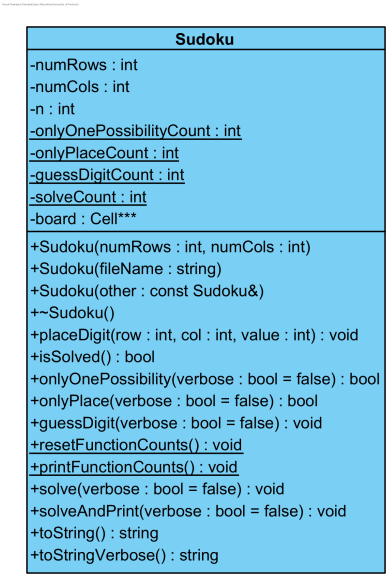


Figure 5: Sudoku UML

4.3.1 Members

- -numRows: int
  - This is the number of rows inside each block.
  - It is also the number of columns of blocks in the grid.
- -numCols: int
  - This is the number of columns inside each block.
  - It is also the number of rows of blocks in the grid.

- -n:int
  - This is the largest value allowed in the grid.
  - It is calculated as $n = \text{numRows} \times \text{numCols}$.
  - This is also how many cells is inside each row and column of the grid.
- -onlyOnePossibilityCount: int
  - This is a static variable used to keep track of how many times the onlyOnePossibility function was called.
  - It should be initialised to 0.
- -onlyPlaceCount: int
  - This is a static variable used to keep track of how many times the onlyPlace function was called.
  - It should be initialised to 0.
- -guessDigitCount: int
  - This is a static variable used to keep track of how many times the guessDigit function was called.
  - It should be initialised to 0.
- -solveCount: int
  - This is a static variable used to keep track of how many times the solve function was called.
  - It should be initialised to 0.
- -board: Cell***
  - This is a 2D dynamic array of dynamic cells. It is used to represent the grid.
  - This array has n rows and n columns.

4.3.2 Functions

- +Sudoku(numRows: int, numCols: int)
  - This is the blank constructor for a Sudoku grid. Initialise all non static member functions. The grid should be initialised such that no cells are filled in. The possibilities array of each cell should have all the values from 1 to n, in ascending order.
  - If n is larger than 15, then throw a SudokuTooBig object. This is done to prevent timing out on Fitchfork due to the complexity associated with the recursive solution.

- +Sudoku(fileName: string)
  - This is the file constructor for a Sudoku grid. It will be used to create partially filled in Sudoku grids.
  - You may assume that the file exists and follows the correct format.
  - You may assume the format of the textfiles will always be the following:
    * Row 1 will have two integers separated by a space. The first number is the number of rows in each block. The second number is the number of columns in each block. You may assume both of these will be positive non zero numbers.
    * Rows 2 to $n + 1$ will each be one row of the Sudoku grid. Cells are separated by spaces. There might be more than one space between cells, to make the textfile more readable. Each cell will either be a non zero positive number or a -. Numbers should be interpreted as a filled in cell. - should be interpreted as a cell that is not filled in.
  - If n is larger than 15, then throw a SudokuTooBig object. This is done to prevent timing out on Fitchfork due to the complexity associated with the recursive solution.
  - Each cell should be filled in to match the textfile's grid. The filled in cells should update the possibilities array of all cells in its row, column and block to follow the rules of Sudoku. *Hint: There is a function in this class that can help with this.*
  - If an IllegalCellValue exception is encountered when reading in the cells, then delete all of the dynamic memory of this object and throw the same exception.
- +Sudoku(other: const Sudoku&)
  - This is the copy constructor for the Sudoku class. **Remember to make deep copies.**
- + ∼ Sudoku()
  - This is the destructor for the Sudoku class.
- +placeDigit(row: int, col: int, value: int): void
  - Call setValue on the cell at the passed-in row and column. You may assume that the passed-in row and column values are inside the grid. If the setValue function throws an exception then throw that exception again.
  - Update the possibilities arrays of all cells in its row, column and block to follow the rules of Sudoku.
- +isSolved(): bool
  - Returns true if the Sudoku is solved, and returns false otherwise.
  - A board is consider solved if all cells are filled in.
  - If any cells are not filled in and also have no possibly values then throw a UnsolvableSudoku object.

- +onlyOnePossibility(verbose: bool = false): bool
  - This function tries to place a digit into the Sudoku grid. If a digit was placed it returns true, otherwise it returns false. A maximum of one digit can be placed by one call to this function.
  - When the function is called increase the static variable keeping track of how many times this function is called.
  - Search for the first cell in the grid with only one possible value. The search should be from left to right, and then from top to bottom. Once a cell is found with only one possible value, fill in that cell using the placeDigit function.
  - If the passed-in verbose variable is true then print out the following message. This message should be on its own line. The 1 should be replaced by the value that was placed. The 2 should be replaced the row it was placed in, and the 3 should be replaced by the column it was placed in:

```
onlyOnePossiblity placed 1 at r2c3
```

- +onlyPlace(verbose: bool = false): bool
  - This function tries to place a digit into the Sudoku grid. If a digit was placed it returns true, otherwise it returns false. A maximum of one digit can be placed by one call to this function.
  - When the function is called increase the static variable keeping track of how many times this function is called.
  - This function places digits for which there is only one place in a row, column or block. First loop through each row and see if inside each row there is only one place to place a certain digit. If multiple digits in a row only has one place, then place the smallest digit. Once you iterated through all rows, then do the same for the columns. Once you are done with the columns, do the same for each block, starting at the top left block and moving right to left, then top to bottom.
  - If the passed-in verbose variable is true then print out the following message. This message should be on its own line. The 1 should be replaced by the value that was placed. The 2 should be replaced the row it was placed in, and the 3 should be replaced by the column it was placed in:

```
onlyPlace placed 1 at r2c3
```

- +guessDigit(verbose: bool = false): void
  - This function will solve the current grid if a solution exists.
  - When the function is called increase the static variable keeping track of how many times this function is called.
  - This method will recursively search for a solution.
  - Loop through the grid from left to right and then top to bottom. Find the cell with the least number of possibilities. The value of this cell will be guessed. Once you found the correct cell do the following:
    * Loop through all the possible values for this cell. Each possible value will be guessed until a solution is found.
    * If the passed-in verbose variable is true then print out the following message. This message should be on its own line. The 1 should be replaced by the value that will be guessed. The 2 should be replaced the row it will be placed in, and the 3 should be replaced by the column it will be placed in:

      ```
      guessDigit trying 1 at r2c3
      ```

    * Create a copy of the current grid using the copy constructor. Call the placeDigit function on the copy using the guess value.
    * Call the solve function on the copy and remember to pass the verbose variable down to the new call.
    * If the solve function throws an UnsolvableSudoku exception, then catch that exception and continue to the next guess, since this guess is wrong.
    * If an exception was not caught that means the guess was correct. Update all values in the current board to match the values of the copy. If the verbose variable is true print out the following on its own line:

      ```
      guessDigit found solution and is backtracking
      ```

    * If you iterate through all possibilities without finding a solution, throw an UnsolvableSudoku object.

- +resetFunctionCounts(): void
  - Set all static member variables to 0.

- +printFunctionCounts(): void
  - Print out the following with each element being on its own line. The 0's should be replaced by the static member values.

    ```
    Function call counts
    onlyOnePossibility: 0
    onlyPlace: 0
    numGuessed: 0
    solve: 0
    ```

- +solve(verbose: bool = false): void
  - This function should implement the flowchart in Figure 6.

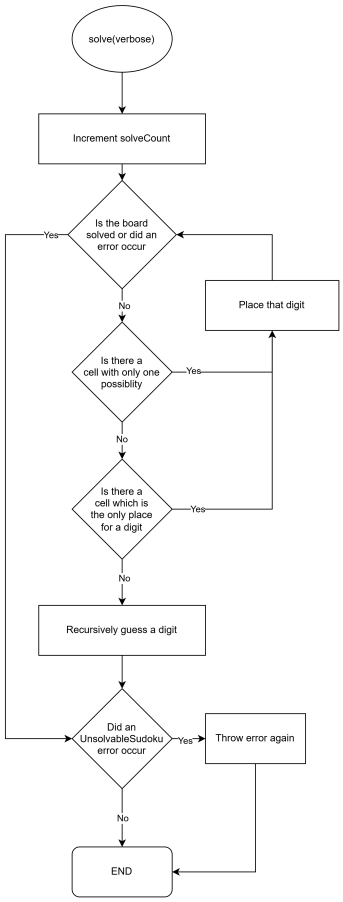


Figure 6: Caption

- +solveAndPrint(verbose: bool = false): void
  - This function should do the following:
    - Call resetFunctionCounts.
    - Call solve(remember to pass down the verbose variable).
    - If verbose is true, print out toStringVerbose, otherwise print out toString.
    - Call printFunctionCounts.
- +toString(): string
  - This function is given to you in the header file. Do not change it.
  - This gives a string representing the grid. Filled in cells will show their value, and cells that are not filled in will show -.
- +toStringVerbose(): string
  - This function is given to you in the header file. Do not change it.
  - This gives a string representing the grid. Filled in cells will show their value, and cells that are not filled in will show the possiblities array.

5 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

6 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the main.cpp file to aid your testing. In order to determine the coverage of your testing the gcov [1] tool, specifically the following version *gcov (Debian 8.3.0-6) 8.3.0*, will be used. The following set of commands will be used to run gcov:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%
40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing mark. Remember that your main will be testing the Instructor Provided code and as such, it can

[1] For more information on gcov please see `https://gcc.gnu.org/onlinedocs/gcc/Gcov.html`

only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

7 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **c++98**.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
  - fstream
  - iomanip
  - iostream
  - sstream
  - string
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

8 Upload Checklist

The following c++ files should be in a zip archive named uXXXXXXXX.zip where XXXXXXXX is your student number:

- Cell.cpp
- Sudoku.cpp
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

9 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (`https://ff.cs.up.ac.za/`). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

and run with the following command:

```
./main
```

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.
You have 5 submissions, and your best mark will be your final mark. Upload your archive to the Assignment 1 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**